

Machine Learning Notes: Batch vs. Online Learning

1. Introduction

In previous lessons, we discussed Machine Learning types based on the **amount of supervision** required (Supervised, Unsupervised, Semi-supervised, and Reinforcement Learning).

Today, we are categorizing Machine Learning based on **how a model is trained** and how it behaves in a **Production Environment**.

2. What is a "Production Environment"?

Before understanding training types, we must understand where the model lives.

- **Development Environment (Local):** This is your own computer or laptop where you write code, experiment, and build the model.
- **Production Environment (Server):** This is the live environment (a server) where your code actually runs for the customers.
 - **The Logic:** When you build software, it shouldn't just stay on your machine. It needs to be deployed to a server with an IP address so customers can access it.
 - **In ML context:** When you deploy your model to a server to handle real-world data, that environment is called "Production."

3. Batch Learning (Offline Learning)

Batch Learning is the **conventional (traditional) way** of training Machine Learning models.

Definition

In Batch Learning, the model is trained using the **entire available dataset** at once.

- **No Incremental Learning:** The model does not learn "on the fly" or in small bits. It takes the whole "batch" of data, learns from it, and then starts working.
- **Offline Nature:** Because processing a massive amount of data requires high computational power and time, this training is usually done **offline** on a data scientist's powerful machine, not on the live production server.

The Workflow (Step-by-Step)

1. **Collect Data:** Gather all the data you have.
2. **Train Model:** Use the entire dataset to train the ML model on your local/offline system.
3. **Test Model:** Verify if the model's performance is accurate.

4. **Deploy to Server:** Once satisfied, upload the "trained model" to the production server.
5. **Run:** The model now sits on the server and provides predictions to users without further learning.

Example 1: Netflix Recommendation Engine

- **Action:** You train a model on all existing movies and user ratings on your machine.
- **Deployment:** You put it on the Netflix server.
- **Result:** It suggests movies to users based on what it learned during training.

Example 2: Spam Filter

- **Action:** You train a classifier on thousands of "Spam" and "Ham" emails.
- **Deployment:** It starts filtering your inbox.

4. The Problem with Batch Learning

The biggest issue is that **Batch Models are Static**.

Why is this a problem?

- **Evolving Business Scenarios:** The world changes. Netflix adds new movies every week. If the model was trained last month, it doesn't know about the new movies.
- **Evolving Tactics:** In a Spam Filter, hackers invent new ways to write spam emails. A static model will eventually become "obsolete" (useless) because it hasn't learned the new patterns.

The Solution: Periodic Retraining

To keep a Batch Model updated, engineers follow a cycle:

1. Collect new data coming into the system.
 2. **Merge** the new data with the old data.
 3. **Retrain** the entire model from scratch on the full updated dataset.
 4. **Replace** the old model on the server with the new one.
- *Frequency:* This can happen every 24 hours, every week, or every month depending on the data size.

5. Disadvantages of Batch Learning

1. Data Size (Big Data Problem)

As the business grows, the data grows. Eventually, the dataset becomes so huge (Terabytes) that:

- Your computer might not have enough RAM/CPU to process the "whole chunk" at once.
- The code might "crash" because it cannot handle the entire batch.

2. Hardware and Connectivity Limitations

Sometimes, models run in places with **no internet** or **limited access**.

- **Example (Army App):** If you make an app for soldiers in remote areas (like Ladakh) where there is no internet, you cannot easily pull the model back, retrain it, and push it back to their devices.
- **Example (Satellites):** Once a satellite is launched, you have very limited connectivity to update its "brain" frequently.

3. Immediate Availability (Trend Sensing)

Batch learning fails when you need to react to a trend **immediately**.

- **The "Demonetization" Example:**
 - Suppose a news app uses Batch Learning and updates every 24 hours.
 - Suddenly, "Demonetization" happens. Everyone is searching for it.
 - The model (trained 10 hours ago) doesn't know this is a trend yet. It won't show you the relevant news until the next 24-hour update.
 - By the time it updates, the news might already be old or irrelevant.

Key Highlights & Tips

- **Highlight:** Batch Learning is also called **Offline Learning** because the training happens away from the live production flow.
- **Tip:** If your data doesn't change much (e.g., predicting the type of a leaf based on its shape), Batch Learning is perfectly fine.
- **Important Point:** In Batch Learning, there is **no learning on the server**. The server only uses the "knowledge" the model already has.

Next Topic: Online Learning (Where the model learns incrementally from live data).